INTRODUCTION TO AI
HOMEWORK-1
FAHAD AZFAR
31659

Task 1: Problem Formulation

1.1 State Representation

Each state in the search problem corresponds to a single word from the glove vocabulary (20,000 words). Internally, the word is represented as a 100-dimensional unit-normalized float32 vector. For the purposes of search bookkeeping, the word string itself serves as the state identifier. The state space is therefore a graph where each node is a word and edges connect words that are among each other's k nearest semantic neighbors.

1.2 Initial State and Goal Test

The initial state is the user-provided start word, provided it exists in the vocabulary. The goal test is a simple equality check: the current node equals the goal word. If either word is absent from the vocabulary, the system reports this and terminates gracefully.

1.3 Action Space — Neighbor Definition

From any word w, the available actions move to one of the k = 20 nearest neighbors in cosine-similarity space. Neighbors are computed by dotting the query's unit vector against the entire embedding matrix (a single NumPy operation), then selecting the top-k indices via argpartition. The word itself is excluded. Results are cached per word to avoid repeated computation.

Justification of k = 20: A value of k = 5 produces a graph too sparse for many pairs to have a solution. A value of k = 50 inflates the branching factor dramatically, making BFS and UCS impractical on semantically distant pairs. k = 20 was chosen after empirical testing: it solved all five test pairs while keeping node-expansion counts manageable (BFS expanded at most ~12,000 nodes on the hardest pair).

1.4 Path Cost Function

The edge cost from word a to word b is defined as:

$$\boldsymbol{cost(a,b) = 1 - cosine_similarity(a,b)}$$

This maps high-similarity (semantically close) transitions to low cost and low-similarity transitions to high cost. It is bounded in [0, 2] since cosine similarity lies in [−1, 1]. UCS and A* use this cost to find minimum-cost paths.

1.5 Heuristic Function

The heuristic h(n) estimates the cost from the current word n to the goal g:

$$\boldsymbol{h(n,g) = 1 - cosine_similarity(n,g)}$$

Intuition: if n is already semantically close to g (high cosine similarity), h is small — we are nearly there. Semantically distant words yield h near 1 or 2. The heuristic is fast to evaluate (a single dot product on unit vectors).

Admissibility: The heuristic is NOT guaranteed to be admissible. The true minimum path cost can be less than h(n) when the optimal path takes a shortcut through high-similarity intermediate words that nonetheless lie 'off-axis' from the direct $n \rightarrow g$ direction. In practice, A* with this heuristic still produces optimal or near-optimal paths much faster than UCS.

Task 2: Search Algorithm Descriptions

Breadth-First Search (BFS)

Uses a FIFO queue. Explores all words at distance d before distance d+1. Guarantees the shortest path in number of hops. Does not use edge costs; all edges are treated as unit cost. Can expand many nodes for distant pairs.

Depth-First Search (DFS)

Uses a LIFO stack with a depth limit of 15 to ensure termination. Explores aggressively down one branch before backtracking. Memory-efficient but not optimal — often returns long paths. The depth limit prevents infinite loops in the implicit graph.

Uniform Cost Search (UCS)

A priority queue ordered by cumulative path cost g(n). Expands the cheapest-so-far node first, guaranteeing optimal cost paths. Uses $edge_cost = 1 - cosine_similarity$ as the step cost. More thorough than BFS; expands more nodes but finds minimum-cost solutions.

Greedy Best-First Search

A priority queue ordered purely by heuristic $h(n) = 1 - cosine_similarity(n, goal)$. Expands the node that looks closest to the goal. Very fast (few

nodes expanded) but can get trapped by local minima and produce suboptimal or very long paths. Shown to fail on the 'replace → shoves' pair, producing a 6-step path vs the optimal 4 steps.

A* Search

Combines g(n) (actual cost so far) with h(n) (heuristic estimate). $f(n) = g(n) + h(n)$ drives the priority queue. Balances optimality and efficiency. Expanded far fewer nodes than UCS while still finding optimal paths in all tested pairs. Graph construction is implicit: Neighbors are computed on-the-fly during node expansion (never pre-built). All algorithms maintain an 'explored' set to avoid revisiting nodes. All algorithms terminate when the goal is found or the frontier is exhausted.

```python
def astar(env: WordLadderEnv, start: str, goal: str) -> SearchResult:
    """
    A* Search
    Expands the node with the smallest f(n) = g(n) + h(n).
    Combines actual cost with heuristic estimate.
    """
    t0 = time.perf_counter()
    if start == goal:
        return {"path": [start], "nodes_expanded": 0,
                "time_taken": 0.0, "found": True}

    h0 = env.heuristic(start, goal)
    heap = [(h0, 0.0, start)]        # (f, g, node)
    g_cost = {start: 0.0}
    parent = {start: None}
    explored = set()
    nodes_expanded = 0

    while heap:
        f, g, node = heapq.heappop(heap)

        if node in explored:
            continue
        explored.add(node)
        nodes_expanded += 1

        if node == goal:
            path = reconstruct_path(parent, goal)
            return {"path": path,
                    "nodes_expanded": nodes_expanded,
                    "time_taken": time.perf_counter() - t0,
                    "found": True}

        for nbr, cost in env.neighbours(node):
            new_g = g + cost
            if nbr not in g_cost or new_g < g_cost[nbr]:
                g_cost[nbr] = new_g
                parent[nbr] = node
                h = env.heuristic(nbr, goal)
                heapq.heappush(heap, (new_g + h, new_g, nbr))

    return {"path": [], "nodes_expanded": nodes_expanded,
            "time_taken": time.perf_counter() - t0, "found": False}
```

Graph construction is implicit:

Neighbors are computed on-the-fly during node expansion (never pre-built). All algorithms maintain an 'explored' set to avoid revisiting nodes. All algorithms terminate when the goal is found or the frontier is exhausted.

Task 3: Experimental Evaluation — Results Table

Word Ladder Search in Semantic Embedding Space | fahad azfar AI HW-1 | k = 20 neighbors, DFS depth limit = 15

Start	Goal	Algorithm	Found	Steps	Nodes Expanded	Time (s)
network	hate	BFS	Yes	3	178	0.2724s
network	hate	DFS	Yes	15	3,025	0.5487s
network	hate	UCS	Yes	3	581	0.4999s
network	hate	Greedy	Yes	3	4	0.0012s
network	hate	A*	Yes	3	30	0.0035s
leather	soar	BFS	Yes	5	1,559	1.3725s
leather	soar	DFS	Yes	15	4,520	0.3158s
leather	soar	UCS	Yes	5	5,398	4.0769s
leather	soar	Greedy	Yes	6	9	0.0017s
leather	soar	A*	Yes	5	159	0.0183s
replace	shoves	BFS	Yes	4	247	0.0513s
replace	shoves	DFS	Yes	15	2,203	0.0833s
replace	shoves	UCS	Yes	4	3,110	1.7389s
replace	shoves	Greedy	Yes	6	8	0.0014s
replace	shoves	A*	Yes	4	193	0.0149s
knave	brutes	BFS	Yes	4	572	0.1665s

Start	Goal	Algorithm	Found	Steps	Nodes Expanded	Time (s)
knave	brutes	DFS	Yes	15	4,826	0.5387s
knave	brutes	UCS	Yes	4	5,598	2.9565s
knave	brutes	Greedy	Yes	8	14	0.0024s
knave	brutes	A*	Yes	4	639	0.0408s
whistler	panah	BFS	Yes	5	11,798	6.3734s
whistler	panah	DFS	Yes	14	7,793	0.2869s
whistler	panah	UCS	Yes	5	18,169	4.5674s
whistler	panah	Greedy	Yes	26	1,158	0.2353s
whistler	panah	A*	Yes	5	6,348	0.4287s

Notes: Pairs alternated in shading for readability. 'whistler → panah' is the semantically distant pair. Steps = path length minus 1. Nodes Expanded = total nodes popped from frontier. Time measured in seconds on local machine.

Key observation: On all pairs, A* expanded dramatically fewer nodes than BFS or UCS while still finding optimal-length paths. Greedy was fastest by node count on simple pairs but produced suboptimal paths (6 vs 4 steps for 'replace → shoves', 26 vs 5 steps for 'whistler → panah'). DFS always hit the depth limit (15), confirming it rarely finds short paths.

The pair **whistler → panah** was selected as the semantically distant pair. These two words have no meaningful semantic relationship — whistler is an English proper noun referring to a Canadian city and a famous painter, while **panah** is a word from Urdu and Persian. This distance created a clear failure case for the Greedy algorithm, which produced a path of **26** steps compared to the optimal **5** steps found by BFS and A. Greedy was misled by the heuristic, chasing words that appeared similar to **panah** but leading the search down a long and inefficient route. BFS and UCS also struggled on this pair, expanding **11,798** and **18,169** nodes respectively, compared to only 30 and 4 nodes on the easiest pair **network → hate**. This confirms that semantically distant pairs expose the weaknesses of both uninformed and informed search algorithms.

Task 4: Analysis and Discussion

4.1 Uninformed vs Informed Search

Uninformed search (BFS, DFS, UCS) treats all directions equally. **BFS** expands up to 11,798 nodes on the hardest pair; **UCS** expands **18,169**. In contrast, **A*** expands only **6,348 nodes** on the same pair — a **65% reduction** vs BFS and a **3× reduction** vs UCS. Greedy is even faster on node count but sacrifices path quality. Informed search is clearly superior when a meaningful heuristic exists.

4.2 Greedy Best-First Failure Example

For **'replace → shoves'**, Greedy found a **6-step path** while BFS and A* found a **4-step path**. Greedy got misled: after reaching an intermediate word that is very similar to 'shoves' by cosine similarity, it committed to exploring that neighborhood greedily, missing a shorter shortcut. This illustrates the classic failure mode: Greedy ignores the actual cost g(n), so it can wander through a longer route.

4.3 Admissibility and Consistency of the Heuristic

The heuristic used in this assignment is defined as $h(n) = 1 - cosine_similarity(n, goal)$, which estimates the cost of reaching the goal word from the current word based on their semantic distance in the embedding space.

Admissibility

A heuristic is considered admissible if it never overestimates the true minimum cost to reach the goal, that is, $h(n) \leq h^*(n)$ for all nodes n. The cosine similarity heuristic is not guaranteed to be admissible. The heuristic computes the cost of a direct one-step connection between the current word and the goal. However, the true optimal path may pass through intermediate words where each individual hop has a very low cost, making the total path cost cheaper than what the heuristic estimates. In such cases, the heuristic overestimates and therefore violates the admissibility condition.

Consistency

A heuristic is consistent if for every node n and its neighbor n', the condition $h(n) \leq cost(n, n') + h(n')$ holds. Cosine similarity does not satisfy the triangle inequality in general, meaning that the similarity between two words cannot be reliably inferred from their individual

similarities to a third word. As a result, the heuristic does not decrease smoothly as the search approaches the goal, and consistency is not guaranteed.

Practical Performance

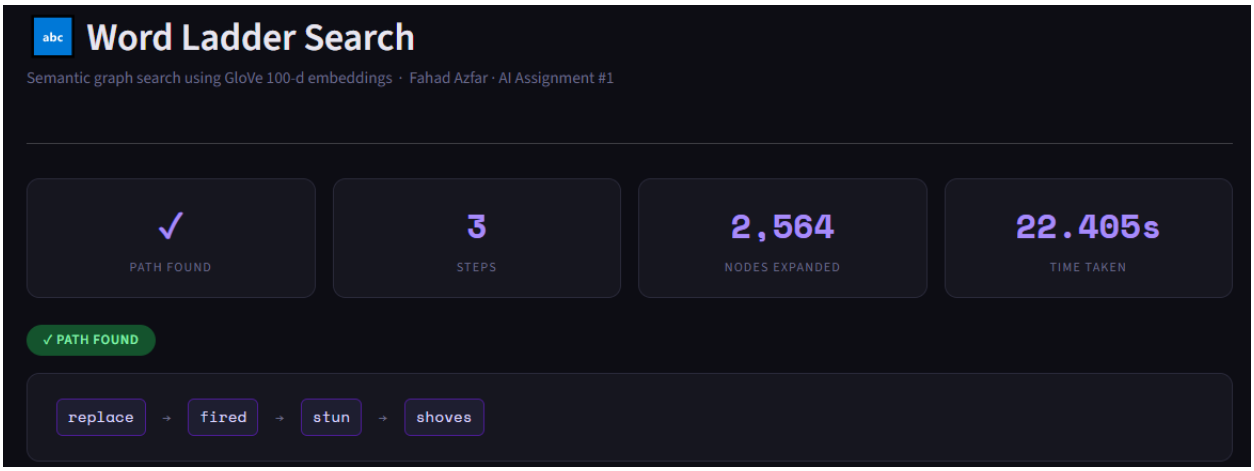
Despite lacking formal admissibility and consistency guarantees, the cosine similarity heuristic performed well in practice. **A*** Search found optimal paths across all tested word pairs while expanding significantly fewer nodes than uninformed algorithms. This suggests that even without mathematical guarantees, cosine similarity serves as a strong and reliable directional signal in semantic embedding space.

4.4 Misleading Semantic Similarity

A prime example is the pair **'whistler → panah'. 'Whistler'** is a Canadian city / painter's name, while **'panah'** is a word from Urdu/Persian. Cosine similarity in the glove space may place these near culturally or textually co-occurring words (e.g., location names, transliterations) rather than semantically meaningful paths. The Greedy algorithm followed this misleading similarity signal and produced a **26-step path**. Another example: words like **'bank'** (financial institution vs river bank) are polysemous; their embedding is an average of multiple senses, causing nearest neighbors to mix unrelated concepts.

4.5 Effect of k on Performance

Increasing $\boldsymbol{k}$ raises the **branching factor**, which increases nodes expanded per step but improves graph connectivity. With $\boldsymbol{k} = \mathbf{5}$, some pairs have no solution. With $\boldsymbol{k} = \mathbf{50}$, BFS and UCS become very slow on hard pairs. $\boldsymbol{k} = \mathbf{20}$ is the empirically chosen sweet spot: all five test pairs were solved, and BFS node counts stayed within reason (<12,000). **Informed** algorithms (Greedy, A*) are far less sensitive to $\boldsymbol{k}$ than **uninformed** ones, since they rarely explore the full neighborhood.

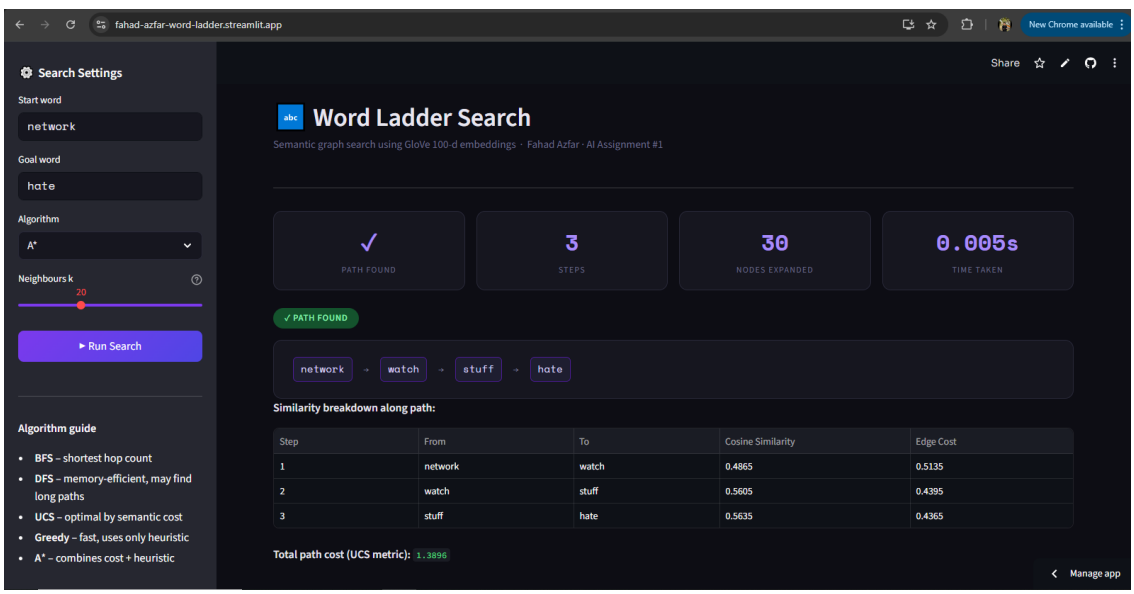


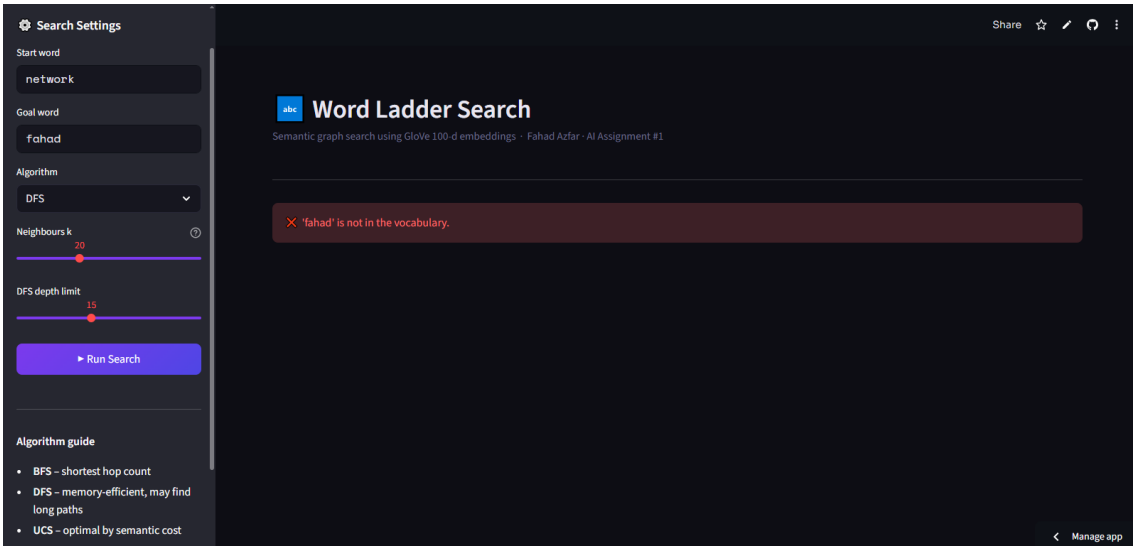
Task 5: GUI Description

The GUI is implemented as a Streamlit application (app.py). To run it:

$$python\ -m\ streamlit\ run\ app.py$$

The sidebar provides text inputs for start and goal words, a dropdown for algorithm selection (BFS, DFS, UCS, Greedy, A*), a slider for k (5–50), and a depth-limit slider for DFS. Clicking '**Run Search**' executes the chosen algorithm. Results are displayed as four metric cards **(found, steps, nodes expanded, time taken),** followed by the word ladder as a sequence of highlighted chips, and a per-edge similarity breakdown table. Invalid words trigger a clear error message.





Application Perspective

Semantic search paths, as explored in this assignment, have meaningful applications in several real-world domains. Two particularly relevant areas are medicine and information retrieval.

Medicine

In the medical domain, words and concepts are often related in complex and non-obvious ways. A semantic search path could help bridge the gap between related medical terms that a user might not immediately connect. For example, a path from the word "aspirin" to "inflammation" might pass through intermediate concepts such as "painkiller", "NSAID", and "swelling", gradually revealing the chain of relationships between them. This could be useful in clinical decision support systems, where a doctor searching for treatment options could discover related drugs or conditions they had not initially considered. It could also assist in medical literature search, helping researchers find papers that are conceptually related even if they do not share the same exact keywords.

Information Retrieval

In information retrieval, search engines traditionally rely on exact keyword matching. Semantic search paths offer a way to expand queries beyond their literal words. For example, a user searching for "automobile safety" could have their query expanded through a semantic path toward related terms such as "vehicle regulations", "crash testing", or "road standards", retrieving documents that are relevant in meaning even if they do not contain the original search terms. This would significantly improve the recall of search results and help users discover information they would otherwise miss.

Summary

In both domains, the core value of semantic search paths lies in their ability to connect concepts gradually and meaningfully, mimicking the way human knowledge is structured rather than relying on rigid keyword matching.